

PROBLEM 1 : Image Classification with CNN + MLP Head

Rajat Vajpayee

1 Setup

MNIST: 28×28 grayscale digits. I split the 60,000 training images into 50,000 for training, 10,000 for validation and kept the official 10,000 test images untouched. All architecture choices were made on the validation set; the test set was used once, for Table 3.

Everything is written by hand using plain PyTorch tensors. No `autograd`, no `nn.Conv2d`, no `nn.Linear`.

Hardware and training time. Experiments ran on a NVIDIA A100-40GB. Batch size 256, learning rate 0.01, 20 epochs. The best model trained in **2 min 59 s** (8.9 s per epoch).

2 Task 1: Architecture, Forward and Backward Pass

Table 1: Best model (Model 4).

Layer	Output	Config	Parameters
Input	$1 \times 28 \times 28$	Grayscale	0
Conv1 + ReLU	$16 \times 28 \times 28$	16 filters, 3×3 , pad 1	160
Max Pool	$16 \times 14 \times 14$	2×2 , stride 2	0
Conv2 + ReLU	$32 \times 14 \times 14$	32 filters, 3×3 , pad 1	4,640
Max Pool	$32 \times 7 \times 7$	2×2 , stride 2	0
Flatten	1568	$32 \times 7 \times 7$	0
FC + ReLU	256	$1568 \rightarrow 256$	401,664
Output	10	$256 \rightarrow 10$ (logits)	2,570
Total parameters			409,034

2.1 Why these choices

16 then 32 filters. Pooling halves the image size, so I double the filter count to keep roughly the same amount of information at each depth. Table 3 supports this: the second conv block gave the biggest jump in the whole experiment.

Max pooling. Shrinks the feature map and makes the MLP computation head cheap.

256 hidden units. Chosen by testing. With one conv block, 64 to 128 units changed nothing. With two conv blocks, 128 to 256 gained 0.27 points. A bigger head only helps once the features feeding it are good enough.

ReLU. It does not saturate, so gradients do not shrink to zero through the stack the way they do with sigmoid or tanh.

2.2 Forward pass

$$P_1 = \text{MaxPool}(\text{ReLU}(W_1 * X + b_1)), \quad P_2 = \text{MaxPool}(\text{ReLU}(W_2 * P_1 + b_2)), \quad (1)$$

$$A_3 = \text{ReLU}(W_3 \text{ Flatten}(P_2) + b_3), \quad Z_4 = W_4 A_3 + b_4, \quad (2)$$

with $p_i = e^{Z_{4,i}} / \sum_j e^{Z_{4,j}}$ and $L = -\frac{1}{N} \sum_i \sum_c y_{ic} \log p_{ic}$.

2.3 Backward pass

Backpropagation was implemented manually by propagating the loss gradient from the output layer to the input layer using the chain rule. For the softmax activation with cross-entropy loss, the gradient with respect to the output logits is

$$\frac{\partial L}{\partial z} = \frac{\hat{y} - y}{N}, \quad W \leftarrow W - \eta \frac{\partial L}{\partial W}, \quad (3)$$

where $\hat{y}$ is the predicted probability vector, y is the one-hot encoded target, N is the mini-batch size, and η denotes the learning rate.

Table 2: Backward propagation through the network.

Layer	Backward computation
Softmax + Cross Entropy	$\frac{\partial L}{\partial z} = \frac{\hat{y} - y}{N}$
Output FC	$\frac{\partial L}{\partial W} = a^\top \frac{\partial L}{\partial z}, \quad \frac{\partial L}{\partial a} = \frac{\partial L}{\partial z} W^\top$
ReLU	$\frac{\partial L}{\partial x} = \frac{\partial L}{\partial y} \odot \mathbf{1}(x > 0)$
Hidden FC	$\frac{\partial L}{\partial W} = x^\top \frac{\partial L}{\partial y}, \quad \frac{\partial L}{\partial x} = \frac{\partial L}{\partial y} W^\top$
Flatten	Reshape gradient to the pooled feature-map dimensions
Max Pool	Gradient routed only to the maximum element in each pooling window
Conv + ReLU	ReLU mask applied, then convolution gradients computed
Convolution	$\frac{\partial L}{\partial W} = x * \frac{\partial L}{\partial y}, \quad \frac{\partial L}{\partial x} = \frac{\partial L}{\partial y} * W$
Parameter Update	$W \leftarrow W - \eta \frac{\partial L}{\partial W}$

3 Task 2: Results

Table 3: Test set results (10,000 held-out images). Precision, recall and F1 macro-averaged.

Model	Filters	Hidden	Prec.	Rec.	F1	Acc.
Baseline	[8]	32	0.9690	0.9689	0.9689	0.9689
Model 1	[16]	64	0.9711	0.9710	0.9710	0.9710
Model 2	[16]	128	0.9693	0.9691	0.9691	0.9691
Model 3	[16, 32]	128	0.9792	0.9790	0.9790	0.9790
Model 4	[16, 32]	256	0.9818	0.9817	0.9817	0.9817

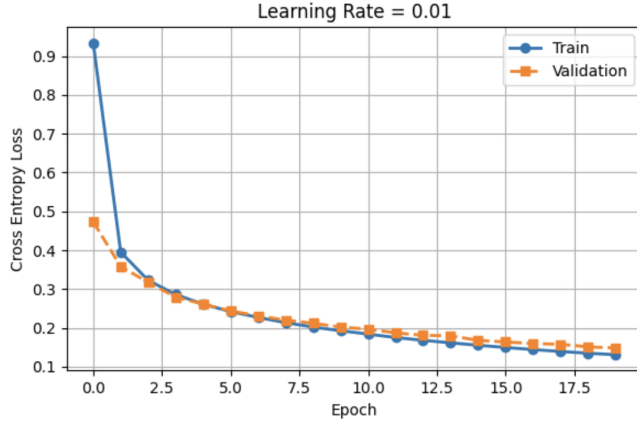
Depth matters. The second conv block gained 0.99 points (2 to 3). Doubling the hidden layer without adding depth gained nothing (1 to 2). Width only helped once the second block was there (3 to 4). In the shallow models the bottleneck was feature quality, not classifier capacity.

Why all four metrics coincide. MNIST classes are near-equal in size (9.0% to 11.2%) and the model errs at a similar rate on each, so macro-averaging reweights almost nothing and precision, recall, F1 and accuracy all land on ≈ 0.982 . Those four numbers therefore carry about one number’s worth of information; the confusion matrix holds the rest.

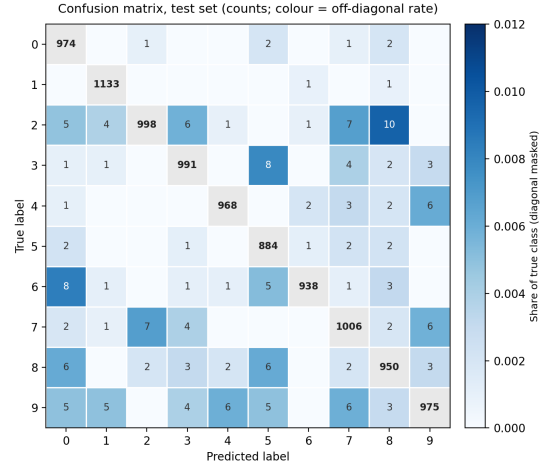
Table 4: Per-class results, Model 4.

Digit	0	1	2	3	4	5	6	7	8	9
Precision	0.970	0.990	0.990	0.981	0.990	0.971	0.995	0.975	0.972	0.982
Recall	0.994	0.998	0.967	0.981	0.986	0.991	0.979	0.979	0.975	0.966
F1	0.982	0.994	0.978	0.981	0.988	0.981	0.987	0.977	0.974	0.974

Where the errors are. The 183 errors are spread thin, not concentrated: the worst single confusion ($2 \rightarrow 8$) is only 5.5% of them. They are also asymmetric : 2 is misread as 8 ten times but 8 as 2 only twice. With no dominant failure mode left, the model is near the limit of what this architecture extracts, so the next gain would have to come from a design change (more depth, augmentation) rather than more training.



(a) Loss vs training steps.



(b) Confusion matrix; colour = off-diagonal rate.

Figure 1: Training loss falls $0.736 \rightarrow 0.046$ and validation $0.374 \rightarrow 0.070$. Validation tracks training closely from epoch 2. So, no overfitting signal was noticed and hence no regularization was added.

4 Task 3: SGD Convergence Conditions

4.1 Ergodicity — holds

I computed the class distribution over all 50,000 training images, then drew 1000 random mini-batches at each of ten batch sizes and measured the KL divergence between the average mini-batch distribution and the dataset distribution.

Table 5: KL divergence between mini-batch and dataset class distributions, 1000 trials each.

Batch size	1	2	4	8	16	32	64	128	256	512
Mean KL	2.303	1.680	1.107	0.656	0.340	0.149	0.072	0.036	0.018	0.009
Std KL	0.049	0.214	0.214	0.191	0.140	0.072	0.035	0.017	0.008	0.004

KL falls steadily with batch size. At $B = 1$ a mini-batch is a single sample, so its class distribution is a one-hot vector and the divergence is at its largest; by $B = 256$, the divergence is down to 0.018, meaning a mini-batch closely matches the dataset distribution.

4.2 Unbiased gradient and bounded variance — both supported

Unbiased gradient. Taking the gradient over all 50,000 training images as the reference, I sampled many random mini-batches at each batch size, averaged their gradients and measured the relative error against the reference across all 6 parameter tensors. The error shrinks steadily as batch size grows (Figure 2a), from about 2.4126 at $B = 1$ to about 0.0086 at $B = 512$: the expected mini-batch gradient matches the full-batch gradient. This holds because the network has no BatchNorm or dropout, so the mini-batch loss is a plain average over samples.

Bounded variance. At fixed initial parameters, I sampled many mini-batches per batch size and estimated the variance of the stochastic gradient (Figure 2b). It is highest at $B = 1$, where few samples contribute to each estimate, falls as more samples are averaged, and is effectively zero at the largest batch sizes. Variance is finite at every batch size tested, which supports the assumption, though measured at one parameter setting, not throughout weight space.

4.3 Step size — Not satisfied verified

I trained the same model at $\eta \in \{0.001, 0.01, 0.03, 0.1\}$ for 20 epochs and compared the training and validation curves (Figure 2c):

- 0.001: stable but slow, and finishes with the lowest accuracy.
- 0.01: fast convergence, with training and validation curves closely matching : stable optimization and good generalization.
- 0.03: slightly faster than 0.01 but a small train-validation gap opens in the later epochs.
- 0.1: fastest initially, but validation accuracy fluctuates between epochs, indicating reduced stability.

Increasing the learning rate improves convergence up to a point; 0.01 gives the best trade-off between speed, stability and validation performance here. However, this experiment does not include the evaluation with continuously varying learning rates. Since a constant learning rate is used, empirical convergence is verified, but the theoretical Robbins-Monro diminishing step-size condition is not satisfied.

4.4 L -smoothness — only locally, and only as a lower bound

L -smoothness requires $\|\nabla f(x) - \nabla f(y)\| \leq L\|x - y\|$ for all x, y , equivalently $\|\nabla^2 f(x)\|_2 \leq L$. Verifying it globally means bounding the largest Hessian eigenvalue over the whole parameter space, which is out of reach for a model with 409,034 parameters.

What can be measured is the constant near the trained solution. Holding a fixed mini-batch, I perturbed all parameters by $\delta = \varepsilon \mathcal{N}(0, I)$ with $\varepsilon = 10^{-4}$, recomputed the gradient, and estimated

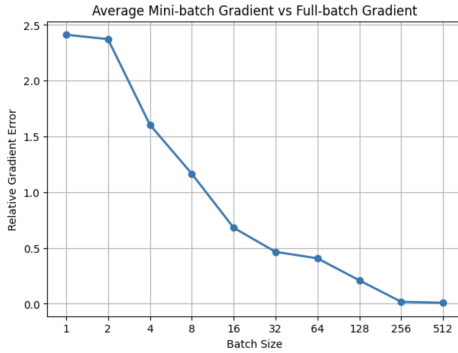
$$\hat{L} = \frac{\|\nabla f(\theta + \delta) - \nabla f(\theta)\|}{\|\delta\|} \quad (4)$$

over 200 random perturbations. The result is finite and tightly grouped: mean 0.273, standard deviation 0.125, range 0.058 to 0.667. So the objective behaves consistently with a locally smooth function around the solution SGD reached.

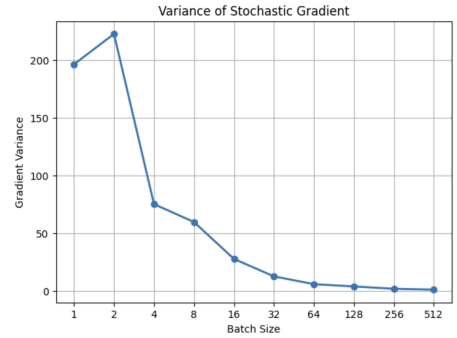
The estimated values remained finite across all perturbations, indicating stable local gradient behavior. Therefore, the optimization objective exhibits empirical evidence consistent with local L -smoothness around the trained solution. However, this experiment does not establish global L -smoothness, which would require bounding the Hessian over the entire parameter space.

4.5 Summary

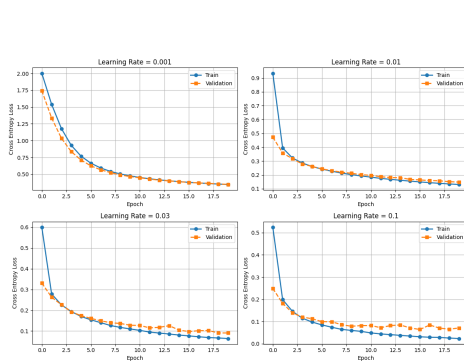
Three conditions are supported empirically, one is not fully satisfied because I used a constant learning rate and one cannot be checked at all with the compute available (due to large Hessian matrix). Training still reaches 98.17% test accuracy, so the empirical evidence points to stable optimization even though the full set of theoretical conditions is not met. (Please refer to the `notebooks/` directory in the GitHub repository: <https://github.com/rajatvajpayee/et-robo.git>.)



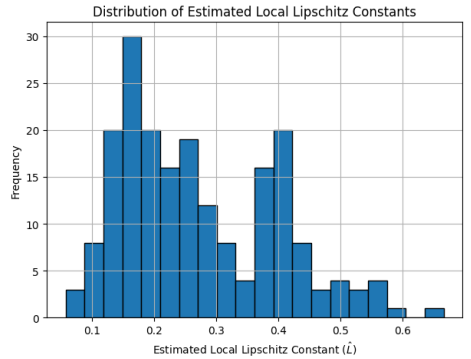
(a) Gradient error vs. batch size.



(b) Gradient variance vs. batch size.



(c) Loss at four learning rates.



(d) Estimated local L -smoothness constant.

Figure 2: Task 3 experiments. (a) Gradient error decreases with batch size, supporting unbiasedness. (b) Gradient variance remains finite and decreases with batch size. (c) Learning-rate comparison showing that $\eta = 0.01$ provides the best balance between convergence speed and stability. (d) Estimated local L -smoothness around the converged solution.